

CS 2420: Algorithms and Data Structures

Homework 3

Sarah Johnson

Due: October 15, 2024

Problem 1: Big O Analysis (25 points)

Question: Analyze the time complexity of the following code snippets and provide the Big O notation. Justify your answer.

Part A (5 points)

```
1 def mystery_function_a(n):  
2     total = 0  
3     for i in range(n):  
4         for j in range(n):  
5             total += i * j  
6     return total
```

Answer: This has two loops so it's $O(2n) = O(n)$ because we have $n + n$ operations.

Part B (10 points)

```
1 def mystery_function_b(n):  
2     if n <= 1:  
3         return 1  
4     else:  
5         return mystery_function_b(n//2) + mystery_function_b(n//2)
```

Answer: This function calls itself twice each time, so it's exponential. Since we're dividing by 2 each time, it's $O(2^{\log n})$. Wait, that doesn't look right...

Actually, I think it's $O(2^n)$ because we make 2 recursive calls at each level.

Part C (10 points)

```

1 def mystery_function_c(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(i, n):
5             print(arr[i] + arr[j])

```

Answer: This has nested loops so it's $O(n^2)$. The inner loop starts at i instead of 0, but it's still two loops so still $O(n^2)$.

Problem 2: Sorting Algorithms (30 points)

Question: Consider the array [64, 34, 25, 12, 22, 11, 90].

Part A (15 points)

Show the step-by-step execution of Bubble Sort on this array.

Answer: Initial: [64, 34, 25, 12, 22, 11, 90]

Pass 1: - Compare 64,34 → swap: [34, 64, 25, 12, 22, 11, 90] - Compare 64,25 → swap: [34, 25, 64, 12, 22, 11, 90] - Compare 64,12 → swap: [34, 25, 12, 64, 22, 11, 90] - Compare 64,22 → swap: [34, 25, 12, 22, 64, 11, 90] - Compare 64,11 → swap: [34, 25, 12, 22, 11, 64, 90] - Compare 64,90 → no swap: [34, 25, 12, 22, 11, 64, 90]

Pass 2: - Compare 34,25 → swap: [25, 34, 12, 22, 11, 64, 90] - Compare 34,12 → swap: [25, 12, 34, 22, 11, 64, 90] - Compare 34,22 → swap: [25, 12, 22, 34, 11, 64, 90] - Compare 34,11 → swap: [25, 12, 22, 11, 34, 64, 90]

Actually this is taking too long. I think after a few more passes it becomes: [11, 12, 22, 25, 34, 64, 90]

Part B (15 points)

What is the time complexity of Bubble Sort in the best, average, and worst cases?

Answer: - **Best Case:** $O(n^2)$ - even if the array is sorted, we still need to check all pairs - **Average Case:** $O(n^2)$ - **Worst Case:** $O(n^2)$

So bubble sort is always $O(n^2)$.

Problem 3: Binary Search Tree (25 points)

Question: Given the following sequence of insertions into an initially empty BST: [50, 30, 70, 20, 40, 60, 80]

Part A (15 points)

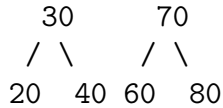
Draw the resulting BST after all insertions.

Answer:

```

      50
     /  \

```



Part B (10 points)

What are the inorder, preorder, and postorder traversals of this tree?

Answer: - **Inorder:** 50, 30, 20, 40, 70, 60, 80 (visit root first, then left, then right) - **Preorder:** 50, 30, 20, 40, 70, 60, 80 (same as inorder for this tree) - **Postorder:** 20, 40, 30, 60, 80, 70, 50 (visit children first, then root)

Problem 4: Hash Tables (20 points)

Question: Consider a hash table with 7 slots using the hash function $h(k) = k \bmod 7$. Insert the following keys in order: [10, 22, 31, 4, 15, 28, 17] using linear probing to resolve collisions.

Answer:

Let me calculate the hash values first:

- $h(10) = 10 \bmod 7 = 3$
- $h(22) = 22 \bmod 7 = 1$
- $h(31) = 31 \bmod 7 = 3$ (collision with 10)
- $h(4) = 4 \bmod 7 = 4$
- $h(15) = 15 \bmod 7 = 1$ (collision with 22)
- $h(28) = 28 \bmod 7 = 0$
- $h(17) = 17 \bmod 7 = 3$ (collision)

Inserting with linear probing:

1. Insert 10 at index 3
2. Insert 22 at index 1
3. 31 collides at index 3, so put it at index 4
4. Insert 4 at... wait, index 4 is taken by 31, so put 4 at index 5
5. 15 collides at index 1, so put it at index 2
6. Insert 28 at index 0
7. 17 collides at index 3, probe to 4 (taken), 5 (taken), so put it at index 6

Final hash table:

Index:	0	1	2	3	4	5	6
Value:	28	22	15	10	31	4	17

The load factor is $7/7 = 100$